

RantLab: Domain Admin Portal & Mobile Alert System

Task Sheet & Engineering Work Breakdown | Domain Portals (Library, Hostel, Mess, IT) • Mobile Push Notifications • Semantic Grouping

1. Feature Overview & Ownership Matrix

This task adds a multi-domain administrator portal with real-time mobile push notifications. When students submit complaints, the AI pipeline automatically categorizes the domain (Library, Hostel, Mess, IT), clusters similar issues, alerts the respective domain administrator on their phone, and provides a unified dashboard to review and resolve grouped complaints.

Member	Role & Files Owned	Core Deliverables for this Feature
Akilan <i>Lead Architect</i>	backend/app/main.py backend/app/llm.py backend/app/models.py docs/api-contract.md	• API Contract & Pydantic models for domain/status • LLM domain extraction prompt & fallback logic • Admin REST endpoints (GET /api/admin/..., PATCH status) • Public resolution feedback on card page
Arvind <i>AI & DB Lead</i>	backend/app/db.py backend/app/cluster.py backend/app/embed.py backend/app/notify.py	• SQLite schema migration (domain, status, subscriptions table) • Domain-scoped clustering engine • Cluster representative & metrics computation • WebPush & Telegram notification dispatcher
Jayasree <i>Frontend Lead</i>	frontend/app/admin/ frontend/components/ frontend/lib/api.ts frontend/lib/types.ts	• Responsive Mobile Admin Portal (/admin/[domain]) • Grouped / Clustered Accordion component • Mobile WebPush permission & subscription prompt • Action controls (In-Progress / Resolve) with toast feedback
Priyan <i>QA & Demo Lead</i>	data/seed.json scripts/ backend/tests/ docs/	• 30+ domain-specific seed complaints across all domains • Automated pytest suite for admin endpoints & clustering • Notification simulation script (test_push_alert.py) • End-to-end demo rehearsal & verification

2. Master Phase Schedule

Phase & Milestone	Owner(s)	Est. Time	Target Output & Checkpoint
Phase 1: Architecture & Contracts	All	1.5h	API contract locked, SQLite schema migrated, seed complaints created, frontend types aligned.
Phase 2: Core Domain Logic & Clustering	Akilan & Arvind	2.5h	LLM prompt extracts domain, cluster engine groups per domain, admin endpoints ready.
Phase 3: Admin UI & Mobile WebPush	Jayasree & Arvind	3.0h	Mobile admin dashboard built, cluster accordions functional, WebPush alerts active.
Phase 4: Feedback Loop, QA & Live Demo	Priyan & All	2.0h	Status updates sync back to students, push alert test script passes, live demo rehearsed.

3. Detailed Task Breakdown by Teammate

■ Akilan — Pipeline, LLM Prompt & API Integration

Owned Files: main.py, llm.py, models.py, docs/api-contract.md

Task & Deliverable	Est.	Technical Details & Implementation Steps	Acceptance Criteria
A1. API Contract & Data Models	1.0h	Define Domain literal ('library' 'hostel' 'mess' 'it_infra' 'academics' 'general') and Status ('new' 'in_progress' 'resolved'). Add AdminCard, AdminCluster, and UpdateStatusPayload schemas.	Models import cleanly; contract diff posted in team chat.
A2. LLM Domain Classification & Urgency	2.0h	Update the fix-generation prompt to output 'domain' and 'urgency' ('low' 'medium' 'high'). Add keyword-based heuristic fallback in STUB_MODE=1 so frontend dev works seamlessly offline.	POST /api/rant returns validated domain tag under 10s.
A3. Admin Endpoints Implementation	2.0h	Create GET /api/admin/{domain}/cards with ?status= filter and cluster metadata. Create PATCH /api/admin/cards/{id}/status to transition complaint lifecycle.	Endpoints return correct filtered JSON and handle invalid domains with HTTP 400/404.
A4. Public Resolution Feedback Loop	1.0h	Update GET /api/card/{id} to return resolution status and optional admin note so students can track progress.	Student card view reflects updated status immediately.

■ Arvind — Database, Clustering & Notification Engine

Owned Files: db.py, cluster.py, embed.py, notify.py

Task & Deliverable	Est.	Technical Details & Implementation Steps	Acceptance Criteria
B1. DB Schema Migration & Subscriptions	1.5h	Add 'domain TEXT', 'status TEXT DEFAULT "new"', and 'resolved_at TIMESTAMP' columns to cards table. Create 'admin_subscriptions' table (id, domain, push_subscription_json, created_at).	Database initializes with new columns without breaking seed loader.
B2. Domain-Scoped Clustering Algorithm	2.0h	Update cluster(items, threshold) to partition embeddings by domain first, ensuring hostel complaints never cluster with library complaints.	Clustering returns domain-specific cluster IDs ('k_hostel_01', 'k_lib_02').
B3. Cluster Representative & Summaries	1.5h	Compute cluster card count, most representative central complaint using most_central(), and aggregate urgency level.	GET /api/admin/{domain}/clusters returns grouped summaries with card counts.
B4. Mobile Push Dispatch Engine	2.5h	Build notification dispatcher using WebPush (pywebpush) with Telegram bot fallback. Trigger alerts on new complaint arrival or cluster threshold >= 3.	Test script successfully sends push payload to registered admin subscription.

Jayasree — Frontend & Mobile Admin Portal UI

Owned Files: frontend/app/admin/, components/, lib/api.ts

Task & Deliverable	Est.	Technical Details & Implementation Steps	Acceptance Criteria
C1. Admin Dashboard Page (/admin/[domain])	2.5h	Build mobile-first admin dashboard with domain switcher, summary stat cards (Active, Grouped, In Progress, Resolved), and status filters.	Dashboard renders on mobile screen with zero horizontal overflow.
C2. Grouped Cluster Accordion View	2.5h	Build expandable accordion cards showing: Cluster Problem Title, Affected Count Badge (e.g. '8 students'), expanded list of rants, and 3 AI Recommended Fixes.	Accordion expands smoothly; displays individual transcripts and timestamps.
C3. Mobile Push Opt-in Component	1.5h	Create 'Enable Mobile Alerts' modal requesting browser Notification & ServiceWorker permissions. Send subscription payload to POST /api/admin/subscribe.	Admins can toggle notifications on/off from mobile browser.
C4. Status Action Controls & Toast Feedback	1.5h	Add quick action buttons: 'Mark In Progress', 'Resolve Cluster', and 'Assign Fix'. Trigger optimistic UI update with instant toast confirmation.	Clicking resolve marks all cards in cluster as resolved with live feedback.

Priyan — QA, Test Data, Simulation & Demo Lead

Owned Files: data/seed.json, scripts/, backend/tests/

Task & Deliverable	Est.	Technical Details & Implementation Steps	Acceptance Criteria
D1. Domain Seed Dataset Creation	1.5h	Create 30+ realistic domain-specific complaints: 8 Hostel (Wi-Fi, water, hygiene), 8 Library (AC, noise, books), 8 Mess (food quality, queues), 6 IT (lab PCs, portal).	load_seed.py populates DB with pre-tagged domains and vectors.
D2. Admin API Automated Test Suite	1.5h	Write pytest test suite covering: domain filtering, cluster isolation across domains, status update transitions, and subscription registration.	pytest backend/tests/ passes with 100% success rate.
D3. Push Alert Simulation Script	1.5h	Create a CLI test script that posts a synthetic complaint, triggers the notification engine, and validates delivery in < 2 seconds.	CLI outputs clear confirmation of notification dispatch.
D4. Live Demo Flow & Rehearsal	1.5h	Prepare the live demo script: 1) Student rants on phone, 2) Library admin receives instant push alert, 3) Admin opens clustered portal, 4) Admin picks AI fix.	Team completes seamless 5-minute rehearsal without blockers.

4. Feature Definition of Done (DoD)

- **Domain Classification:** Every complaint is accurately tagged to one of: Library, Hostel, Mess, IT, Academics, or General.
 - **Domain Clustering:** Similar complaints cluster together only within their respective domain (no cross-domain grouping).
 - **Mobile Notifications:** Domain admins receive an immediate phone push alert when a new complaint is filed or when a cluster reaches \$lge 3\$ complaints.
 - **Admin Portal Experience:** Admins can filter by status (New, In Progress, Resolved) and expand clusters to see all individual rants and AI fixes.
 - **Feedback Loop:** Resolving an issue in the admin portal automatically updates the status seen by students on their public card view.